

01 / STELLENWERTE · CA. 12 MINUTEN

Tag 4 · Gezielt festigen

Name: _____

Datum: _____

Bearbeite die Aufgaben der Reihe nach. Plane etwa 60 Minuten ein. Schreibe die Rechenwege auf. Nutze Python erst für die Kontrolle auf Seite 4. Wenn du im Cheatsheet nachschlägst, markiere die betreffende Aufgabe. Alle signed Zahlen sind Zweierkomplementzahlen. Ergänze bei Bedarf ein separates Blatt.

1. Genau umrechnen

a) Stelle die Dezimalzahl 78 mit genau acht Bits dar. Schreibe zuerst die verwendeten Stellenwerte auf. Wandle das Muster dann in zwei Hexziffern um. Prüfe dein Ergebnis durch Rückrechnung von Hex nach Dezimal.

b) Wandle 0xC6 in genau acht Bits und anschließend in eine unsigned Dezimalzahl um. Zeige die beiden Vierergruppen und die Stellenwerte.

2. Bitbreite ist nicht der Zahlenwert

a) Wie viele verschiedene Muster gibt es mit fünf Bits? Welches ist das kleinste und welches das größte Muster? Gib den unsigned Zahlenbereich an.

b) Gib den signed Zahlenbereich für fünf Bits an. Berechne außerdem 2^{8-1} und erkläre, wofür dieser Wert beim Dekodieren von acht Bits steht.

c) Ein Datenwort enthält 24 Bits. Wie viele Bytes und wie viele Hexziffern sind das? Schreibe jeweils die Division auf. Wiederhole für 40 Bits.

02 / SCHRIFTLICH RECHNEN · CA. 20 MINUTEN

Addition und Subtraktion

3. Überträge sichtbar machen

Addiere schriftlich. Markiere jeden Übertrag und rechne anschließend zur Kontrolle in Dezimal. Beide Operanden gelten jeweils als signed Acht-Bit-Zahlen. Gib vollständige Binärsumme, acht gespeicherte Bits, gespeicherten signed Wert, Carry-out und signed Overflow an. Begründe die beiden Flags jeweils kurz.

a) 0110 1110 + 0010 1101

b) 1000 1000 + 1110 1100

4. Subtraktion als Addition

Berechne $23 - 41$ **mit acht Bits** und dokumentiere jeden Schritt:

1. Stelle +23 und +41 mit jeweils acht Bits dar.
 2. Bilde aus dem Muster für +41 das Muster für -41. Zeige beide Schritte.
 3. Addiere +23 und -41 schriftlich mit Überträgen.
 4. Behalte acht Bits und interpretiere sie als signed Zahl.
 5. Prüfe durch die Dezimalrechnung. Entsteht signed Overflow? Begründe.
-

03 / GRENZFÄLLE UND PYTHON · CA. 15 MINUTEN

Die Bedeutung jedes Schrittes

5. Zwei Darstellungsgrenzen

a) Erweitere das signed Fünf-Bit-Muster 10110 auf acht Bits, ohne seinen Wert zu verändern. Nenne das neue Muster und seinen signed Wert. Welcher Wert entstünde stattdessen, wenn du drei Nullen voranstellst?

b) Nenne die kleinste signed Vier-Bit-Zahl und ihr Muster. Invertiere die Bits und addiere 1. Welches Muster entsteht? Warum kann das mathematisch positive Gegenstück nicht in dieser Bitbreite dargestellt werden?

6. Textbearbeitung und Reste

a) Notiere den Wert und den Typ jeder Variablen nach der jeweiligen Zeile. Schreibe Texte mit Anführungszeichen. Erkläre zusätzlich, was die letzte Zeile mit dem vorhandenen Text macht.

```
a = hex(14)
b = a[2:]
c = b.upper()
d = (9 + 3) // 4
e = c.zfill(d)
```

b) Berechne die vier Ausdrücke ohne Python. Begründe die ersten beiden über die Rechenreihenfolge und die letzten beiden über den Divisionsrest. Schreibe für die Modulo-Ausdrücke eine Gleichung der Form „Dividend = ganzzahliges Vielfaches des Divisors + Rest“ auf.

```
(16 + 3) // 4
16 + 3 // 4
19 % 16
-5 % 16
```

04 / TRANSFER UND KONTROLLE · CA. 13 MINUTEN

Papier und Python verbinden

7. Die Funktion gedanklich ausführen

Notiere ohne Python für jeden Aufruf die folgenden Zwischenwerte: `minimum`, `maximum`, `summe`, `overflow`, `raw` und `ergebnis`. Gib dann das Rückgabepaar an. Zeige auch das gespeicherte Vier-Bit-Muster. Nutze die Funktion aus deiner Abschlussdatei als Vorlage.

```
add_with_overflow_flag(3, -7, 4)
add_with_overflow_flag(-8, -2, 4)
```

Zusatzfrage: Was muss bei `add_with_overflow_flag(8, 0, 4)` passieren? Erkläre, warum sich dieser Fall von einem Overflow der Summe gültiger Operanden unterscheidet.

8. Prüfen statt nur ausführen

a) Öffne im Tag-4-Ordner Python und importiere aus `hardware_numbers.abschluss_tag04` die Funktionen `to_hex`, `from_twos_complement` und `add_with_overflow_flag`. Prüfe damit deine Umrechnungen und Additionsresultate aus diesem Blatt. Markiere Abweichungen und korrigiere den ursprünglichen Rechenweg sichtbar.

b) Schreibe in einer neuen Datei `test_festigung.py` genau vier gezielte Tests für deine Abschlussfunktionen:

1. Führende Nullen bei der Hexausgabe.
2. Dekodierung genau an der Grenze zwischen nichtnegativ und negativ.
3. Addition mit negativer Summe und signed Overflow.
4. Ungültiger signed Operand, geprüft mit `pytest.raises(ValueError)`.

Notiere die erwarteten Ergebnisse vor dem Ausführen. Starte aus dem Tag-4-Ordner `python -m pytest -q test_festigung.py` und halte fest, wie viele Tests bestanden haben. Erkläre bei einem Fehler, was du geändert hast.

9. Das Warum in eigenen Worten

Eine CPU kann Subtraktionsbefehle ausführen. Erkläre in zwei Sätzen, warum Zweierkomplement trotzdem hilfreich ist, um Subtraktion mit einer Addierschaltung umzusetzen. Beziehe dich auf deine Rechnung $23 - 41$.
